

Histórico



SQL (Structured Query Language) ou **Linguagem de Consulta Estruturada** é uma linguagem para definição, acesso, manipulação e controle de Bases de Dados Relacionais.

Baseada na **Álgebra Relacional**, foi desenvolvida originalmente no início dos anos 70 nos laboratórios da IBM, dentro do projeto System R, que tinha por objetivo demonstrar a viabilidade da implementação do modelo relacional proposto por **E. F. Codd**.

A linguagem se tornou um grande **padrão** de banco de dados, devido a sua eficácia, simplicidade e facilidade de uso.

4

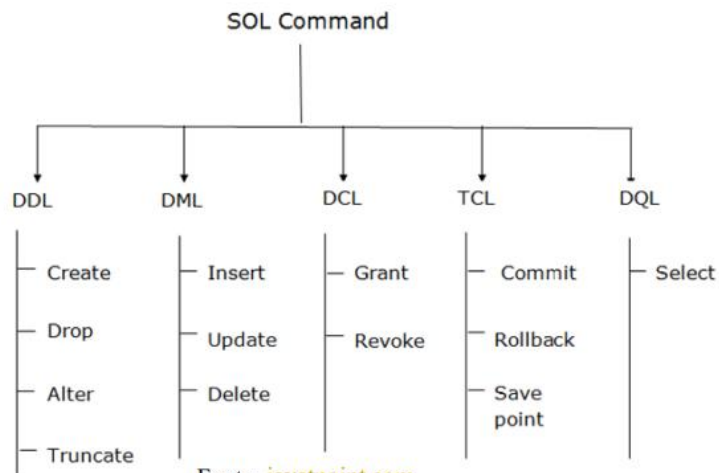
Divisões da SQL

A SQL é dividida em grupos de comandos, de acordo com a sua natureza:

- **DDL** → Data Definition Language (Linguagem de Definição de Dados) Reúne os comandos de criação dos objetos de Bancos;
- **DML** → Data Manipulation Language (Linguagem de manipulação de dados) Comandos utilizados para realizar inclusões, alterações e exclusões dos dados;
- **DQL** → Data Query Language (Linguagem de Consulta de Dados) Agrupa os comandos que permitem ao usuário especificar uma consulta
- **DCL** → Data Control Language (Linguagem de Controle de Dados). Comandos de controle de acesso que atribuem permissões aos usuários.
- **DTL** → Data Transaction Language (Linguagem de Transação de Dados). Controla uma sequência de comandos e a integridade dos dados resultantes.

6

Divisões da SQL



Fonte: javatpoint.com

Obs: Esta divisão é apenas uma convenção. Alguns autores descartam a DQL e colocam o comando **SELECT** como pertencendo à **DML**.

Em caso de concursos ou provas, verifique na bibliografia da banca, qual a convenção utilizada.

7

Constraints



Na criação das tabelas, precisaremos definir os **tipos de dados e restrições (CONSTRAINTS)** para os campos e tabelas.

As **constraints** são regras, validações impostas aos elementos para promover a integridade dos dados. Essas restrições podem ser definidas a nível de campo ou de tabela. Qualquer tentativa de violação dessas regras resultará em erro e a operação não será concretizada.

13

Constraints

- **PRIMARY KEY (PK ou Chave Primária)** → Especifica qual coluna ou conjunto de colunas será a chave primária da tabela (Campo que identifica de forma única um registro na tabela).
- **FOREIGN KEY (FK ou Chave Estrangeira)** → Determina que uma coluna ou conjunto de colunas é uma chave estrangeira e que, portanto, se relaciona a uma PK de outra tabela.
- **UNIQUE** → Determina que o valor de um campo não poderá se repetir dentro da tabela. Será criado um índice para esse campo. Veja mais sobre índices na outra apostila [aqui](#).
- **CHECK** → Especifica uma condição que a coluna precisa atender para poder salvar o registro (a partir da versão 8 do MySQL);
- **NOT NULL** → Determina que a coluna tem preenchimento obrigatório;
- **DEFAULT** → Especifica um valor padrão para gravar no campo, caso este não seja informado no comando de inserção.

14

Considerações sobre NULL e DEFAULT

O valor **NULL** (nulo) é especial em banco de dados e, muitas vezes difícil de ser tratado. Ele será gravado em um campo, quando não for informado nenhum valor para o mesmo durante a inserção (Comando **INSERT**).

Já, a restrição **DEFAULT** permite que seja definido um valor padrão para o campo que será usado para evitar a gravação do **NULL**.

Dessa forma, quando não informamos um valor para o campo, na inserção, o Banco de Dados, verificará o seguinte:

1. Caso exista um valor **DEFAULT**, este será gravado no campo;
2. Caso não exista o **DEFAULT** e o campo for **NOT NULL**, ocorrerá um erro. Caso contrário será gravado o valor **NULL** no campo.

15

A chave primária (Primary Key)

Quando definimos uma chave primária, as seguintes restrições serão impostas:

1. A tabela só pode possuir uma chave primária;
2. O campos da chave não podem conter **NULL**, ou seja, são obrigatórios;
3. O valor da chave será único em toda tabela, ou seja, não pode se repetir;
4. Será criado, por padrão, um índice **PRIMARY**. Veja mais sobre índices na outra apostila [aqui](#).
5. Uma chave primária pode ser composta, ou seja, formada por mais de um campo. Dessa forma, as regras acima valem para a combinação dos dois campos.

Na prática, a restrição PRIMARY KEY é uma combinação de NOT NULL e UNIQUE.

16

Chave estrangeira e Integridade Referencial

Em um banco de dados relacional, quando um registro de uma tabela aponta para outro em outra tabela, é necessário estabelecer regras para que o registro "pai" não possa ser alterado ou excluído se ele tiver "filhos".

Exemplo: Na tabela **departamento**, não podemos excluir um departamento se algum funcionário estiver cadastrado (trabalhando) nele.

Os tipos mais comuns de restrições de integridade são:

CASCADE → Ao se remover um registro da tabela referenciada pela chave estrangeira, os registros relacionados àquele removido serão eliminados em todas as tabelas relacionadas.

RESTRICT → Não permite a remoção de registros que possuam relacionamentos em outras tabelas. (Comportamento padrão)

DEFAULT → Atribuem esses valores para as chaves estrangeiras cujos registros relacionados foram excluídos.

17

Tipos de Dados

O **tipo de dado** de uma coluna define uma faixa de valores válidos (domínio) que podem ser armazenados nessa coluna. Define também as operações passíveis de serem realizadas com esses valores.

O tipo de dados não é considerado uma **constraint**, mas possui a mesma função, pois estabelecemos uma restrição aos valores que serão aceitos e, caso essa regra seja violada, será emitida uma mensagem de erro.

Com certeza é mais uma forma de buscarmos garantir a **integridade** dos dados.

Podemos dividir os tipos de dados do MySQL em 3 categorias:

- **String (Texto)** → Conjunto de valores alfanuméricos (Letras, números e caracteres especiais)
- **Numérico** → Números inteiros e decimais de diversas precisões
- **Data** → Data e Horários que podem ser manipulados em vários formatos

18

Tipos de Dados Numéricos

- **INT, INTEGER** → Inteiros regulares
- **BIGINT, SMALLINT e TINYINT** → Inteiros grandes, pequenos e muito pequenos
- **FLOAT/DOUBLE** → Números de ponto flutuante de precisão simples ou dupla
- **DECIMAL/NUMERIC** → Números de ponto fixo que possuem melhor precisão. Podem ser usados para valores monetários, por exemplo.

Podemos definir um tamanho para cada campo. Caso não o façamos, o sistema assumirá o tamanho padrão.

Exemplos:

INT(5) → Define um campo inteiro com 5 dígitos

DECIMAL(8,2) → Define um número com 6 dígitos inteiros e 2 decimais

19

Variações dos Tipos Numéricos

Para os campos numéricos, podemos ainda definir as seguintes propriedades:

UNSIGNED → Determina que o campo não vai aceitar valores negativos. Útil para aumentar a faixa de valores válidos de números positivos.

AUTO_INCREMENT → O valor do campo será gerado automaticamente de forma sequencial (1,2,3,...). Ou seja, na inclusão, o valor desse campo não precisa ser informado, pois será atribuído o próximo da sequência. Caso seja informado, a sequência continuará a partir desse número.

ZERO_FILL → Campos numéricos serão preenchidos com zeros à esquerda

20

Tipos de Dados Texto

CHAR → Texto de comprimento fixo

VARCHAR → Texto de comprimento variável

TEXT/BLOB → Textos Grandes/Binários

VARCHAR é preferível em relação ao **CHAR**, pois ocupa menos espaço.

Exemplo: Seja um campo **NOME** definido como **VARCHAR(100)**. Ao armazenar o valor 'Zé' nesse campo, ele vai ocupar por volta de 2 bytes. Caso o tipo do campo fosse **CHAR**, ocuparia por volta de 100 bytes.

No entanto, o tipo **CHAR** é mais rápido nas consultas.

Existem ainda os tipos **NCHAR** e **NVARCHAR** que são semelhantes, mas aceitam valores **UNICODE**.

21

Tipos de Dados para Datas

DATE → Datas no padrão YYYY-MM-DD

DATETIME → Data e horário no padrão YYYY-MM-DD HH:MM:SS

TIME → Horário no padrão HH:MM:SS

YEAR → Um ano de 4 dígitos na faixa de 1901 até 2155.

TIMESTAMP → Padrão UTC. Armazena os segundos desde '1970-01-01 00:00:00' UTC

Os campos DATE e DATETIME podem armazenar datas com valores entre 01/Jan/1000 até 31/Dez/9999. Já, o TIMESTAMP até '2038-01-09 03:14:07'.

No MySQL estão disponíveis várias funções para manipulação e formatação de datas que veremos no desenrolar do curso

22

Variações

GENERATED → São campos calculados a partir de outros valores e, portanto, seus valores são gerados automaticamente no momento da inserção/atualização do registro.

ENUM → Tipo de dados **TEXTO** que permite definir um conjunto de dados discretos como domínio do campo.

Exemplo:

Podemos definir o campo **SEXO** como **ENUM('F','M')**

O Banco de dados vai armazenar o valor 1 para masculino e 2 para Feminino. Economiza espaço em disco, mas cuidado, pois o banco ordena esses campos pelo seu índice.

23

Comando INSERT

O comando **INSERT** insere registros nas tabelas, atribuindo conteúdo em seus campos de acordo com o tipo e tamanho dos dados. Dessa forma, é necessário conhecer a estrutura da tabela que se deseja manipular.

Sintaxe:

Nome da tabela que vai receber o novo registro

Nomes dos campos que receberão os valores

```
INSERT INTO <tabela> [( <atributo>, ... )] VALUES  
( expressão, ... );
```

Valores que serão inseridos. A expressão deve ser compatível com o campo respectivo

34

INSERT - Exemplos

```
INSERT INTO departamento  
  (depto_id, depto_nome)  
VALUES  
  (1, 'COMPRAS')
```

Observações:

- Neste exemplo, será inserido um novo registro na tabela **departamento**, sendo que o campo **depto_id** receberá o valor **1** e o campo **depto_nome**, o valor **'COMPRAS'** respectivamente.
- O campo **depto_nome** é do tipo **TEXTO**. Por isso, o valor inserido deve possuir o delimitador **ASPAS SIMPLES**. Isso não ocorre para o campo **depto_ID**, que é numérico.
- Os campos e valores envolvidos devem ser compatíveis quanto ao tipo e tamanho dos dados. Caso contrário, ocorrerá um erro.

35



A **DQL** engloba os comandos que permitem a realização de consultas à base de Dados. Ou seja, o **SELECT**!

Com o comando **SELECT** é possível pesquisar dados em uma ou mais tabelas de acordo com o critério de filtro definido. É o astro da **SQL** e o mais complexo comando da linguagem, como veremos a seguir...

SELECT – Maiúsculas e Minúsculas

SQL não é sensível ao caso, portanto escrever **SELECT** ou **select** ou **SeLeCt** é a mesma coisa. No entanto, o mesmo não vale para o conteúdo dos campos que nem sempre é sensível ao caso. Isso depende do **CHARSET** e **COLLATION** usados. Dessa forma, 'ANTONIO' pode ser diferente de 'antonio'.

Para fazer pesquisas **não** sensíveis ao caso, usamos as seguintes funções:

UPPER() → Converte as letras para maiúscula

LOWER() → Converte as letras para minúsculas

Exemplos: Os seguintes comando produzem o mesmo resultado:

- **SELECT * FROM funcionario WHERE
UPPER(func_nome) = 'JOSE DA SILVA'**
- **SELECT * FROM funcionario WHERE
LOWER(func_nome) = 'jose da silva'**

SELECT – Cláusula DISTINCT

A cláusula **DISTINCT** é usada para eliminar do resultado da consulta todas as linhas duplicadas.

Exemplo: Como listar todos os números de departamentos que possuem funcionários alocados?

Precisamos fazer um **SELECT** na tabela **funcionario** trazendo os departamentos, pois assim garantimos que um funcionário está alocado no departamento. Uma solução poderia ser o seguinte comando:

```
SELECT depto_id FROM funcionario
```

No entanto, esse comando trará vários departamentos repetidos. Para resolver, usamos a cláusula **DISTINCT** da seguinte forma:

```
SELECT DISTINCT depto_id FROM funcionario
```

Obs: A cláusula **DISTINCT** deve vir logo após o comando **SELECT** e se aplica a todas as colunas selecionadas.

64

SELECT – Cláusula LIMIT

A cláusula **LIMIT** é usada para limitar o número de registros retornados. É útil para fazer relatórios paginados ou para testes em tabelas grandes, além de outras finalidades.

Exemplo1: Busca os três primeiros funcionários:

```
select * from funcionario limit 3
```

Exemplo2: Busca os três primeiros funcionários a partir do segundo registro:

```
select * from funcionario limit 2,3
```

65

SELECT – CASE

A expressão **CASE** é uma estrutura de controle que permite adicionar lógica **if-else** a uma consulta **SELECT**. Ela permite escolher um valor entre um conjunto de valores.

```
CASE value
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ...
  [ELSE else_result]
END
```

Exemplo: Exibindo o sexo por extenso.

```
SELECT func_nome,
  CASE func_sexo
    WHEN 'F' THEN 'feminino'
    ELSE 'Masculino'
  END AS Sexo
FROM funcionario
```

Obs:

É possível também usar expressões lógicas para os valores

66

SELECT – Funções de Agregação

As funções de agregação permitem a execução de operações matemáticas e estatísticas sobre um conjunto de dados, retornando um único valor totalizador.

As principais funções de agregação em MySQL são:

MIN → Valor Mínimo

MAX → Valor Máximo

AVG → Média Aritmética

SUM → Soma

COUNT → Contagem

As funções de agregação recebem como parâmetro o nome do campo que vai ser totalizado e, com algumas exceções, desconsideram na totalização os valores NULL.

67

SELECT – Funções de Agregação – Exemplos

Contar o número total de registros da tabela de departamentos:

```
select count(depto_id) from departamento
```

Calcular a média de salários dos funcionários:

```
select avg(func_salario) from funcionário
```

Calcular a média de salários dos funcionários do departamento 1:

```
select avg(func_salario) from funcionario  
where depto_id = 1
```

Calcular a soma de salários e o menor salário do departamento 1

```
select sum(func_salario), min(func_salario)  
from funcionario where depto_id = 1
```

Use **DISTINCT** para ignorar valores duplicados na agregação:

```
select avg(DISTINCT func_salario) from funcionário
```